



IFC TO GRAPH

Extracting Structure from IFC Models

Professor Wassim Jabi, PhD

WHY?

- Moving from geometry → relationships
- IFC is already a rich relational database
- Graphs as a computational abstraction for analysis

IFC AS A RELATIONAL DATABASE

- IFC structure:
 - Entities (walls, slabs, spaces, doors)
 - Relationships (IfcRel*)
- Examples:
 - IfcRelContainedInSpatialStructurey
 - IfcRelConnectsElements
 - IfcRelSpaceBoundary
- These define an implicit graph already embedded in IFC

TWO GRAPH EXTRACTION STRATEGIES

- A. Semantic Graph (IFC-native relationships)
- B. Spatial Graph (geometry-derived relationships)
- Complementary, not competing
- Choice depends on:
 - Data quality
 - Intended analysis
 - Computational constraints

A. GRAPH FROM IFC RELATIONSHIPS

- Extract graph directly from IFC relational entities (IfcRel*)
- Nodes:
 - IFC elements (IfcWall, IfcSpace, etc.)
- Edges:
 - Explicit relationships (RelConnects, RelAggregates, etc.)
- Dictionaries:
 - Store IFC attributes (GUID, type, properties)

A. SUPPORTED IFC RELATIONSHIP TYPES:

- IfcRelConnectsPorts
- IfcRelConnectsPortToElement
- IfcRelAggregates
- IfcRelNests
- IfcRelAssignsToGroup
- IfcRelConnectsPathElements
- IfcRelConnectsStructuralMember
- IfcRelContainedInSpatialStructure
- IfcRelFillsElement
- IfcRelSpaceBoundary
- IfcRelVoidsElement
- IfcRelDefinesByProperties
- IfcRelAssociatesMaterial
- IfcRelDefinesByType

A. IMPLEMENTATION IN TOPOLOGICPY

- Infer relationships using `Graph.ByIFCPath`
- Parse IFC via ifcopenshell
- For each relationship:
 - Identify Relating and Related objects
 - Create edge between corresponding vertices
- Result:
 - Graph mirrors IFC semantic structure

A. ADVANTAGES

- Computationally efficient:
 - Complexity $\approx O(R)$ where R = number of relationships
- No geometric computation required
- Preserves:
 - Hierarchy (building $\rightarrow$ storey $\rightarrow$ space)
 - Connectivity semantics
- Robust to geometric imperfections

A. LIMITATIONS

- Depends on IFC authoring quality
- Missing or incorrect relationships → incomplete graph
- No geometric awareness:
 - Cannot detect adjacency if not explicitly encoded

B. GRAPH FROM SPATIAL RELATIONSHIPS

- Infer relationships using `Graph.BySpatialRelationships`
- Nodes:
 - Topologic elements (cells, faces, etc.) derived from `Topology.ByIFCPath`
- Edges:
 - Derived from spatial predicates (OGC / DE-9IM spatial logic):
 - Touches
 - Overlaps
 - Within
 - Contains
 - intersects

B. COMPUTATIONAL COMPLEXITY

- Pairwise comparison: $O(n^2)$ for n elements
- This becomes prohibitive for large IFC models (10^4 – 10^5 elements)

B. ADVANTAGES

- Independent of IFC data quality
- Captures actual spatial conditions
- Detects:
 - Missing adjacencies
 - Clash conditions
 - Implicit spatial logic

B. LIMITATIONS

- Computationally expensive without optimisation
- Sensitive to:
 - Tolerance issues
 - Geometric inaccuracies
- Requires robust geometry kernel

B. PERFORMANCE OPTIMISATION STRATEGIES

- Axis-Aligned Bounding Boxes (AABB)
 - Fast pre-filter
 - Reject non-intersecting pairs in $O(1)$

B. PERFORMANCE OPTIMISATION STRATEGIES

- Axis-Aligned Bounding Boxes (AABB)
 - Fast pre-filter
 - Reject non-intersecting pairs in $O(1)$
- Bounding Volume Hierarchies (BVH):
 - Tree structure over geometry
 - Query complexity:
 - Build: $O(n \log n)$
 - Query: $O(\log n + k)$
- Spatial Partitioning
 - Grids / Octrees / KD-trees
 - Reduce comparisons to local neighbourhoods

B. PERFORMANCE OPTIMISATION STRATEGIES

Aspect	IFC Relationships	Spatial Relationships
Source	Semantic	Geometric
Complexity	$O(R)$	$O(n^2)$ -> optimised
Reliability	Depends on IFC source	Depends on geometry kernel
Speed	Faster	Slower
Insight	Intended design	Actual spatial conditions

HYBRID STRATEGY

- Combine both approaches:
 1. Start with IFC relationships
 2. Augment with spatial relationships
 3. Validate inconsistencies
- Result:
 - Rich, robust, multi-layered graph

ATTRIBUTED GRAPHS IN TOPOLOGICPY

- Nodes and edges store dictionaries:
 - IFC properties
 - Geometry-derived metrics
 - Analysis results
- Enables:
 - Storage in graph databases (eg Neo4J, Kuzu)
 - Multi-domain queries
 - Graph ML pipelines
 - Visualisation (Plotly / PyVista)

TOWARD GRAPH-BASED DESIGN

- Graphs are not just analysis tools, they can generate design
- They enable:
 - GraphML: Learning from precedent datasets, Prediction/Classification
 - GraphRAG: Constraint-based synthesis

THANK YOU

- Wassim Jabi
- wassim.jabi@gmail.com
- topologic.app